

Configuring BIND DNS Server & Securing Apache

Tom Goncin
July 20th, 2026
ENSE 353
Lab 5

Table of Contents

OBJECTIVE	(What did we do?)	3
STEPS	(How did we do it?)	4
Part 1:	System Update & BIND Installation	4
Part 2:	Local Host Mapping & Port Testing	5
Part 3:	IPv4 & Firewall Configuration	6
Part 4:	Configuring BIND (named.conf) for Public Access	8
Part 5:	Setting Up the Zone File	9
Part 6:	Apache Installation and Virtual Hosts	11
Part 7:	Securing Apache with Certificates	12
OUTCOME	(Why did we do it?)	14

OBJECTIVE

(What did we do?)

Architecting a secure, public-facing web infrastructure on my Rocky Linux server located at the University of Regina.

This lab, “*Configuring BIND DNS Server & Securing Apache*” is configured through accessing the server’s terminal from another location; which was previously set up to access remotely in the predecessor labs.

This process invoked here involves transitioning from the local machine name resolution to configuring a fully operational Berkeley Internet Name Domain (BIND) DNS server. Furthermore, this lab aims to establish secure web traffic by configuring Apache virtual hosts and bypassing local firewall restrictions to manually provision and deploy verified SSL/TLS certificates.

Note:

Due to the University of Regina’s firewall, certain aspects of the output were deemed a security risk and not provided, although the process did complete the process as expected.

STEPS (How did we do it?)

Part 1: System Update & BIND Installation

1. Log in remotely via the browser at and access the Terminal.

<https://142.3.24.32:9090>

2. Switch to the root user by running

`su -`

3. Upgrade system packages using

`dnf upgrade --refresh`

4. Reboot the machine with

`shutdown -r now`

5. Install the BIND DNS server and troubleshooting utilities by running

`dnf install bind bind-utils`

6. Start the daemon with

`systemctl start named`

7. Enable it to start on boot with

`systemctl enable named`

8. Verify its status is running using

`systemctl status named`

```
[copernicus@ed489-ense04 ~]$ su -
Password:
Last login: Mon Aug  3 16:19:09 CST 2026 on pts/0
[root@ed489-ense04 ~]# dnf install bind bind-utils
Last metadata expiration check: 0:04:12 ago on Mon 03 Aug 2026 04:19:48 PM.
Package bind-32:9.18.33-15.el10 2.2.x86_64 is already installed.
Package bind-utils-32:9.18.33-15.el10 2.2.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
complete!
[root@ed489-ense04 ~]# systemctl start named
[root@ed489-ense04 ~]# systemctl enable named
[root@ed489-ense04 ~]# systemctl status named
• named.service - Berkeley Internet Name Domain (DNS)
   Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-08-03 16:22:52 CST; 1min 39s ago
 Invocation: b5dc4fcd53e45e3815f489e7b45c363
    Main PID: 1030 (named)
      Tasks: 26 (limit: 47892)
     Memory: 12.3M (peak: 14M)
        CPU: 78ms
    CGroup: /system.slice/named.service
            └─1030 /usr/sbin/named -u named -c /etc/named.conf -4

Aug 03 16:22:52 localhost.localdomain named[1030]: network unreachable resolving './NS/IN': 170.247.170.2#53
Aug 03 16:22:52 localhost.localdomain named[1030]: network unreachable resolving './DNSKEY/IN': 192.5.5.241#53
Aug 03 16:22:52 localhost.localdomain named[1030]: network unreachable resolving './NS/IN': 192.5.5.241#53
Aug 03 16:22:52 localhost.localdomain named[1030]: network unreachable resolving './DNSKEY/IN': 192.58.128.30#53
Aug 03 16:22:52 localhost.localdomain named[1030]: network unreachable resolving './NS/IN': 192.58.128.30#53
Aug 03 16:22:52 localhost.localdomain named[1030]: network unreachable resolving './NS/IN': 192.33.4.12#53
Aug 03 16:22:52 localhost.localdomain named[1030]: resolver priming query complete: failure
Aug 03 16:22:52 localhost.localdomain named[1030]: managed-keys-zone: Unable to fetch DNSKEY set '.': failure
Aug 03 16:22:52 localhost.localdomain systemd[1]: Started named.service - Berkeley Internet Name Domain (DNS).
Aug 03 16:22:53 localhost.localdomain named[1030]: listening on IPv4 interface enp0s31f6, 142.3.24.32#53
[root@ed489-ense04 ~]#
```

Part 2: Local Host Mapping & Port Testing

1. Create an alias to simplify text editor commands

```
echo alias "emacs='emacs -nw -color=never'" >> ~/.bashrc
```

2. Navigate to the local configurations

```
cd /etc
```

3. Open the local hosts file

```
emacs hosts
```

4. *Context:* System checks the **hosts** file first for name resolution before querying a DNS server.

This allows us to manually assign names to IPs (e.g., mapping a public IP to a name like **craigPublic**) without needing to remember the exact address.

5. Test specific ports using Telnet: (*HTTP*)

```
telnet craigPublic 80
```

and (*MariaDB*)

```
telnet craigPublic 3306
```

6. *Context:* If the connection fails, it indicates the target machine's firewall is successfully blocking public access to that specific port.

Part 3: IPv4 & Firewall Configuration

1. Navigate to the system configuration directory using

```
cd /etc/sysconfig
```

2. Open the file using

```
emacs named
```

3. Force BIND to use standard IPv4 packets by adding the line to the bottom of the file.

```
OPTIONS="-4"
```

```
root@ed489-ense04:/etc/sysconfig

File Edit Options Buffers Tools Conf Help
# BIND named process options
# ~~~~~
#
# OPTIONS="whatever" -- These additional options will be passed to named
#                      at startup. Don't add -t here, enable proper
#                      -chroot.service unit file.
#
# NAMEDCONF=/etc/named/alternate.conf
#                      -- Don't use -c to change configuration file.
#                      Extend systemd named.service instead or use this
#                      variable.
#
# DISABLE_ZONE_CHECKING -- By default, service file calls named-checkzone
#                          utility for every zone to ensure all zones are
#                          valid before named starts. If you set this option
#                          to 'yes' then service file doesn't perform those
#                          checks.
OPTIONS="-4"
```

Context

When running the network diagnostic tools, the system often attempts to resolve using IPv6 first.

- Because certain local ISPs do not support this, and Rocky Linux 10 can struggle to automatically fall back to IPv4, explicitly forcing the system to use standard IPv4 packets prevents connection timeouts.

4. Save and exit, then restart the service using

```
systemctl restart named
```

5. Permit incoming DNS traffic through the firewall by executing

```
firewall-cmd --permanent --zone=public --add-service=dns
```

```
[root@ed489-ense04 ~]# ~cd /etc/sysconfig
-bash: ~cd: command not found
[root@ed489-ense04 ~]# cd /etc/sysconfig
[root@ed489-ense04 sysconfig]# emacs named
[root@ed489-ense04 sysconfig]# systemctl restart named
[root@ed489-ense04 sysconfig]# firewall-cmd --permanent --zone=public --add-service=dns
Warning: ALREADY_ENABLED: dns
success
[root@ed489-ense04 sysconfig]#
```

6. Apply the firewall changes with

```
firewall-cmd --reload
```

7. Verify the rules are active using

```
firewall-cmd --list-all
```

```
[root@ed489-ense04 sysconfig]# firewall-cmd --list-all
public (default, active)
  target: default
  ingress-priority: 0
  egress-priority: 0
  icmp-block-inversion: no
  interfaces: enp0s31f6
  sources:
  services: cockpit dhcpv6-client dns http https ssh
  ports: 44444/tcp
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
[root@ed489-ense04 sysconfig]#
```

Part 4: Configuring BIND (named.conf) for Public Access

1. Navigate to the configuration folder using

```
cd /etc
```

2. Create a backup of the original configuration by running

```
cp named.conf /root/origconfigs/
```

3. Open the configuration file with

```
emacs named.conf
```

4. Update the listening ports to accept traffic from anywhere by changing the local lines to

```
listen-on port 53 { any; };
```

and

```
listen-on-v6 port 53 { any; };
```

5. Allow incoming queries from the internet by updating the query line to

```
allow-query { any; };
```

6. Disable open public recursion by changing the recursion line to

```
recursion no;
```

Context

Open recursion is a major security vulnerability. Leaving it enabled on a public-facing interface allows malicious external actors to bounce requests off the server to execute DNS amplification attacks against other devices.

- We disable it globally but make an exception for 127.0.0.1 so the local server can still resolve addresses for itself as shown next.

7. Make an exception for local recursive lookups by adding

```
allow-recursion { 127.0.0.1; ::1; };
```

8. Add your domain as the primary authority by appending the following block at the bottom of the file:

```
zone "tom.ursse.org" IN {  
    type master;  
    file "tom.ursse.org";  
    allow-update { none; };  
};
```


Part 5: Setting Up the Zone File

1. Navigate to the root directory using
`cd /root`
2. Download the instructor's template using
`wget gelowitz.org/lab.txt`
3. Copy the file into the named directory and rename it for your domain using
`cp lab.txt /var/named/tom.ursse.org`
4. Navigate to the directory with
`cd /var/named`
5. Open the file using
`emacs tom.ursse.org`
6. Replace all instances of the lab test domain with
`tom.ursse.org`
7. Update the Serial tracker to
`2026072001`
8. Update the relevant IP addresses at the bottom of the file to your lab machine's IP,
`142.3.24.32`

```
root@ed489-ense04:/var/named
```

```
File Edit Options Buffers Tools Table Org Text Help
```

```
$TTL 600
```

```
@ IN SOA labtest.ursse.org. root.labtest.ursse.org. (
```

```
    2026072001 ;Serial
```

```
    3600       ;Refresh
```

```
    1800       ;Retry
```

```
    604800     ;Expire
```

```
    600        ;Minimum TTL
```

```
)
```

```
; Specify nameserver and mail server
```

```
    IN      NS      labtest.ursse.org.
```

```
    IN      MX      10 labtest.ursse.org. ; Primary Mail Exchanger
```

```
; Resolve names
```

```
@ IN A 142.3.24.32
```

```
www IN A 142.3.24.32
```

```
mail IN A 142.3.24.32
```

```
someother IN A 142.3.24.32
```

9. Restart the DNS server using

`systemctl restart named`

10. Test the configuration using

`dig @localhost tom.ursse.org`

```
[root@ed489-ense04 named]# dig @localhost tom.ursse.org
;; communications error to ::1#53: connection refused
;; communications error to ::1#53: connection refused
;; communications error to ::1#53: connection refused

; <<>> DiG 9.18.33 <<>> @localhost tom.ursse.org
; (2 servers found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 52658
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags::; udp: 1232
; COOKIE: ca87a6cd35643947010000006a7118f080cd2f41c65dbdbb (good)
;; QUESTION SECTION:
;tom.ursse.org.                IN      A

;; ANSWER SECTION:
tom.ursse.org.                600     IN      A      142.3.24.32

;; Query time: 1 msec
;; SERVER: 127.0.0.1#53(localhost) (UDP)
;; WHEN: Mon Aug 03 16:40:48 CST 2026
;; MSG SIZE rcvd: 86
```

and

`dig @localhost www.tom.ursse.org`

```
[root@ed489-ense04 named]# dig @localhost www.tom.ursse.org
;; communications error to ::1#53: connection refused
;; communications error to ::1#53: connection refused
;; communications error to ::1#53: connection refused

; <<>> DiG 9.18.33 <<>> @localhost www.tom.ursse.org
; (2 servers found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 7669
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags::; udp: 1232
; COOKIE: 33ce1fe0eaa19a94010000006a711928cad3b8076b2fa1be (good)
;; QUESTION SECTION:
;www.tom.ursse.org.            IN      A

;; ANSWER SECTION:
www.tom.ursse.org.            600     IN      A      142.3.24.32

;; Query time: 0 msec
;; SERVER: 127.0.0.1#53(localhost) (UDP)
;; WHEN: Mon Aug 03 16:41:44 CST 2026
;; MSG SIZE rcvd: 90
```

Part 6: Apache Installation and Virtual Hosts

1. Navigate to the Apache configuration directory using
`cd /etc/httpd/conf`
2. Install the required packages using
`dnf install epel-release mod_ssl`
3. Back up the HTTP configuration by running
`cp httpd.conf /root/origconfigs/`
4. Open the configuration using
`emacs httpd.conf`
5. Add the following virtual host blocks directly beneath the Listen 80 line:

```
<VirtualHost *:80>
    DocumentRoot "/var/www/html"
    ServerName "tom.ursse.org"
    Redirect permanent / https://tom.ursse.org/
</VirtualHost>

<VirtualHost *:80>
    DocumentRoot "/var/www/html/mywww"
    ServerName www.tom.ursse.org
    Redirect permanent / https://www.tom.ursse.org/
</VirtualHost>

<VirtualHost *:80>
    DocumentRoot "/var/www/html/roundcube"
    ServerName "mail.tom.ursse.org"
    Redirect permanent / https://mail.tom.ursse.org/
</VirtualHost>
```
6. Restart Apache using
`systemctl restart httpd`
7. Verify it is running with
`systemctl status httpd`

```
[root@ed489-ense04 private]# httpd -t
Syntax OK
[root@ed489-ense04 private]# systemctl restart httpd
[root@ed489-ense04 private]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Drop-In: /etc/systemd/system/httpd.service.d
            └─php-fpm.conf
   Active: active (running) since Mon 2026-08-03 17:04:44 CST; 5s ago
 Invocation: 2b7a7a18f65b4569b50590f8bf025c7e
    Docs: man:httpd.service(8)
   Main PID: 5835 (httpd)
   Status: "Started, listening on: port 443, port 80"
     Tasks: 177 (limit: 47892)
    Memory: 16M (peak: 16.9M)
       CPU: 95ms
   CGroup: /system.slice/httpd.service
           └─5835 /usr/sbin/httpd -DFOREGROUND
             └─5836 /usr/sbin/httpd -DFOREGROUND
               └─5837 /usr/sbin/httpd -DFOREGROUND
                 └─5838 /usr/sbin/httpd -DFOREGROUND
                   └─5847 /usr/sbin/httpd -DFOREGROUND
```

Part 7: Securing Apache with Certificates

1. Install the Certbot software by running

```
dnf install certbot python3-certbot-apache
```

2. Run the automated certificate tool using `certbot --apache` and follow the prompts.

```
certbot --apache
```

```
[root@ed489-ense04 private]# certbot --apache
Saving debug log to /var/log/letsencrypt/letsencrypt.log

Which names would you like to activate HTTPS for?
We recommend selecting either all domains, or all domains in a VirtualHost/server block.
-----
1: tom.ursse.org
2: mail.tom.ursse.org
3: www.tom.ursse.org
-----
Select the appropriate numbers separated by commas and/or spaces, or leave input
blank to select all options shown (Enter 'c' to cancel):
```

3. Navigate to the private TLS directory using

```
cd /etc/pki/tls/private
```

4. Download the instructor's private key using

```
wget gelowitz.org/privkey.pem
```

5. Navigate to the certs directory using

```
cd ../certs/
```

6. Download the instructor's full chain certificate using

```
wget gelowitz.org/fullchain.pem
```

7. Navigate to the configuration directory using

```
cd /etc/httpd/conf.d
```

8. Open the SSL configuration using

```
emacs ssl.conf
```

9. Change the certificate file location to

```
fullchain.pem
```

and the key file location to

```
privkey.pem
```

10. Restore the SELinux security contexts so Apache is permitted to read the newly downloaded certificate files:

```
restorecon -rv /etc/pki/tls/private/
```

```
restorecon -rv /etc/pki/tls/certs/
```

11. Run a syntax check to ensure the configuration is clean before starting the engine:

```
httpd -t
```

12. Restart Apache to apply all secure certificate changes using

```
systemctl restart httpd
```

OUTCOME

(Why did we do it?)

Successfully completed:

- Diagnosed why the automated Certbot failed (*the university firewall blocking Let's Encrypt*).
- Manually pulled the verified .pem certificates into the correct directories.
- Satisfied Rocky Linux's strict SELinux security policies.
- Got Apache actively serving your domain over a secure Port 443 connection.

By completing these objectives above, the server successfully was transformed into an authoritative DNS name server capable of securely handling external queries that will be used in future labs.

Diagnosing the automated Certbot timeout that was caused by the university firewall blocking Let's Encrypt's external servers demonstrated the critical necessity of understanding manual certificate procurement in enterprise environments which was done previously.

Manually migrating the verified .pem certificates into the correct directories, along with active SELinux policy management, ensured that Apache was cleared to actively and securely serve the domain over Port 443.